

INFORME TÉCNICO: DESPLIEGUE Y BASTIONADO DE INFRAESTRUCTURA DE RED CORPORATIVA Y ENTORNOS SEGUROS



ÍNDICE GENERAL

1. Introducción y Objetivos

2. Arquitectura de Red y Topología

- 2.1. Segmentación de Zonas y Zonas de confianza
- 2.2. Componentes de Laboratorio
- 2.3. Reglas de Filtrado Perimetral y NAT (Port Forwarding)

3. Bastionado del Servidor Web (*Hardening* de Nginx)

- 3.1. Ocultación de Firmas (*Server Tokens*) y Cabeceras HTTP Defensivas

4. Despliegue de la Estrategia de Defensa en Profundidad

- 4.1. HIPS a Nivel de Host (Fail2ban)
- 4.2. WAF a Nivel de Aplicación (ModSecurity + OWASP CRS)
- 4.3. IPS a Nivel Perimetral (Suricata en pfSense)

5. Auditoría, Resultados y Conclusiones

- 5.1. Validación de Pruebas de Concepto (PoC)
- 5.2. Evaluación Global y Líneas Futuras

ÍNDICE DE IMÁGENES

1. Configuración de red de las máquinas
2. Configuración red de pfSense
3. Regla NAT
4. Block private networks
5. Configuración “tokens”
6. Configuración headers
7. Verificación versión oculta
8. Filtro web
9. Configuración cárcel
10. Ataque hydra
11. Comprobación de bloqueo
12. Configuración WAF en nginx
13. Ataques rechazados
14. Puerto 80 abierto
15. IP bloqueada por Suricata

1. INTRODUCCIÓN Y OBJETIVOS

El presente documento detalla el diseño, despliegue, auditoría y bastionado (*hardening*) de una infraestructura de red corporativa virtualizada. El proyecto tiene como objetivo principal poner en práctica los conocimientos técnicos correspondientes a la Administración de Sistemas Informáticos en Red (ASIR) y la Especialización en Ciberseguridad, aplicando el principio de Defensa en Profundidad (*Defense in Depth*).

A través de esta simulación práctica se busca:

- Diseñar y segmentar una topología de red segura mediante firewalls perimetrales y Zonas Desmilitarizadas (DMZ).
- Implementar y asegurar servicios de red esenciales (servidor web Nginx) siguiendo guías de bastionado reconocidas.
- Desplegar mecanismos defensivos a nivel de host (HIPS - Fail2ban) y de aplicación (WAF - ModSecurity) para la mitigación autónoma de ataques.
- Simular escenarios de auditoría técnica (escaneo de vulnerabilidades y ataques web) para validar las políticas de filtrado.
- Implementar prevención de intrusiones perimetral (IPS - Suricata) para aislar amenazas en la capa de red.

2. ARQUITECTURA DE RED Y TOPOLOGÍA

La infraestructura del laboratorio se ha desplegado sobre un entorno virtualizado, haciendo uso de redes internas aisladas para garantizar que el tráfico generado durante los ejercicios de auditoría permanezca confinado.

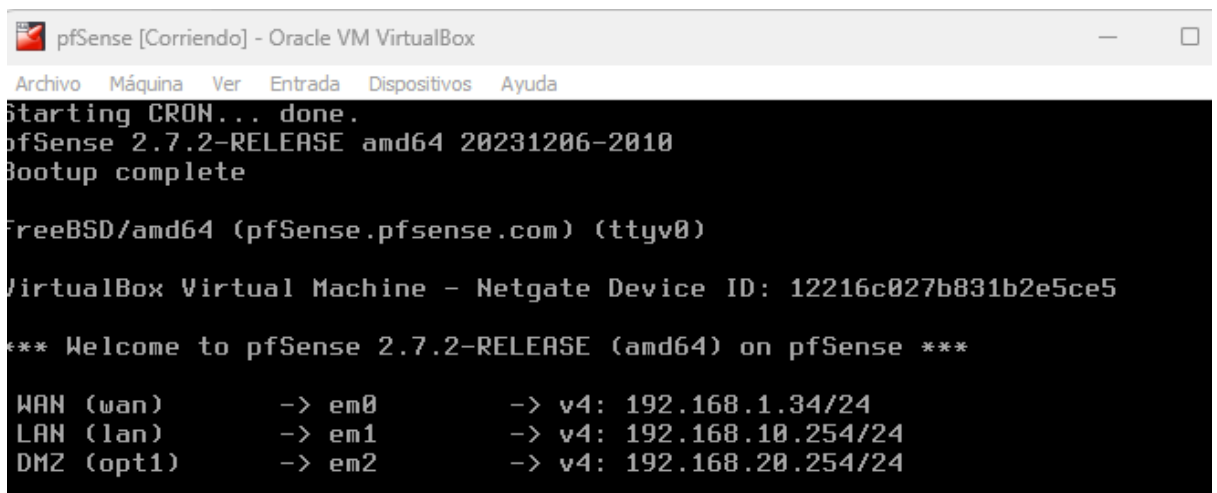
 U Server Apagada	 Red Adaptador 1: Intel PRO/1000 MT Desktop (Red interna, «DMZ_Net»)
 Lubuntu Apagada	 Red Adaptador 1: Intel PRO/1000 MT Desktop (Red interna, «LAN_Corp»)
 Kali_Wan Apagada	 Red Adaptador 1: Red paravirtualizada (Adaptador puente, «TP-Link Wireless USB Adapter»)
 pfSense Apagada	 Red Adaptador 1: Intel PRO/1000 MT Desktop (Adaptador puente, «TP-Link Wireless USB Adapter») Adaptador 2: Intel PRO/1000 MT Desktop (Red interna, «LAN_Corp») Adaptador 3: Intel PRO/1000 MT Desktop (Red interna, «DMZ_Net»)

Imagen 1: Configuración de red de las máquinas

2.1. Segmentación de Redes y Zonas de Confianza

La topología se articula en torno a un firewall perimetral pfSense, dividida en las siguientes zonas:

- **Zona WAN (Red Externa):** Interfaz conectada a la red externa simulada. Actúa como origen de las auditorías externas.
- **Zona DMZ (Zona Desmilitarizada):** Segmento semi-expuesto destinado a alojar servicios de acceso público (Servidor web nginx). Las políticas de filtrado impiden que cualquier host de la DMZ inicie conexiones no solicitadas hacia la LAN.
- **Zona LAN (Red Corporativa Interna):** Segmento aislado y protegido destinado a la administración del sistema.



```
pfSense [Corriendo] - Oracle VM VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
Starting CRON... done.
pfSense 2.7.2-RELEASE amd64 20231206-2010
Bootup complete

FreeBSD/amd64 (pfSense.pfsense.com) (ttyv0)

VirtualBox Virtual Machine - Netgate Device ID: 12216c027b831b2e5ce5

*** Welcome to pfSense 2.7.2-RELEASE (amd64) on pfSense ***

WAN (wan)      -> em0          -> v4: 192.168.1.34/24
LAN (lan)      -> em1          -> v4: 192.168.10.254/24
DMZ (opt1)     -> em2          -> v4: 192.168.20.254/24
```

Imágen 2: Configuración red de pfSense

2.2. COMPONENTES DEL LABORATORIO

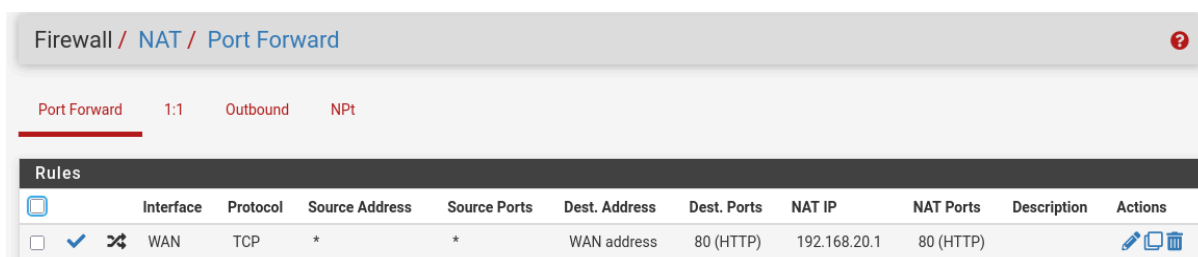
Componente / Host	Sistema Operativo	Rol / Función Principal	Segmento de Red
pfSense	FreeBSD / pfSense	Firewall perimetral, Router, NAT y Suricata (IPS)	WAN / LAN / DMZ
USever	Ubuntu Server	Servidor Nginx, ModSecurity (WAF) y Fail2ban (IPS)	DMZ (192.168.20.1/24)

Kali_Wan	Kali Linux	Estación de auditoría y pruebas de penetración	WAN / Externa (192.168.1.33/24)
Lubuntu	Lubuntu	Interfaz gráfica ligera para acceder a la configuración web de pfSense	LAN (192.168.10.1/24)

2.3. Reglas de Filtrado Perimetral y NAT (Port Forwarding)

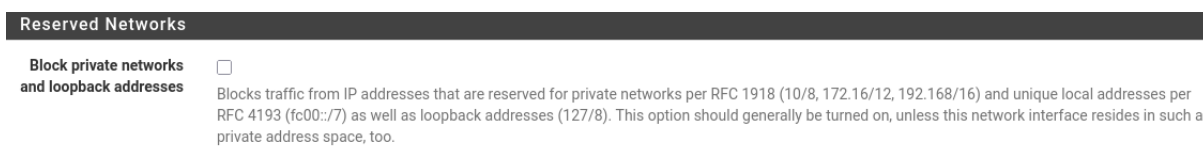
Para permitir la exposición controlada del servidor web de la DMZ hacia el exterior sin comprometer la seguridad del resto de la red, se implementó una regla de redirección de puertos o *Port Forwarding* en el cortafuegos pfSense.

- **Configuración de NAT:** Se estableció una regla en la interfaz WAN para interceptar el tráfico entrante dirigido al puerto 80/TCP (HTTP) y reenviarlo hacia la IP interna del servidor Ubuntu (192.168.20.1) ubicado en la DMZ.



Imágen 3: Regla NAT

- **Regla de Firewall Asociada:** La creación de la regla NAT generó automáticamente una política de acceso en la WAN que permite exclusivamente el tráfico hacia el puerto 80/TCP del objetivo, manteniendo la política general de denegar para el resto de puertos y protocolos.
- **Ajustes de Interfaz WAN:** Para evitar que el cortafuegos bloqueara el tráfico simulado proveniente del hipervisor, se deshabilitó temporalmente la opción "*Block private networks and loopback addresses*" en la configuración de la interfaz WAN.



Imágen 4: Opción "Block private networks..."

3. BASTIONADO DEL SERVIDOR WEB (HARDENING DE NGINX)

Con el objetivo de reducir la superficie de ataque del servidor web expuesto, se procedió a implementar configuraciones de seguridad defensiva directamente en la capa de aplicación (nginx), previniendo vulnerabilidades comunes asociadas a la recolección de información.

3.1. Ocultación de Firmas (Server Tokens) y Cabeceras HTTP Defensivas

Durante la auditoría inicial de reconocimiento con la herramienta `curl` desde la red externa, se detectó que el servidor exponía información detallada sobre su versión y sistema operativo subyacente.

- **Ocultación de Versión:** Se editó el archivo de configuración principal de Nginx (`/etc/nginx/nginx.conf`) para habilitar la directiva `"server_tokens off;"`. Esta acción garantiza que las respuestas del servidor omitan el número de versión, dificultando la labor de perfilado por parte de posibles atacantes.

```
http {  
    ##  
    # Basic Settings  
    ##  
  
    sendfile on;  
    tcp_nopush on;  
    types_hash_max_size 2048;  
    server_tokens off;
```

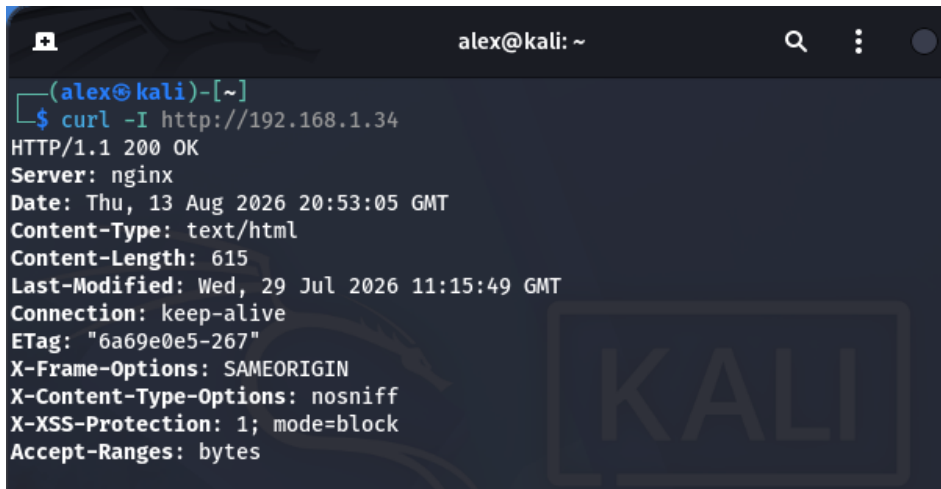
Imagen 5: Configuración tokens

- **Inyección de Cabeceras de Seguridad:** En la configuración del sitio virtual (`/etc/nginx/sites-available/default`), se añadieron cabeceras de respuesta HTTP defensivas para instruir a los navegadores cliente sobre las políticas de seguridad a aplicar:
 - **X-Frame-Options:** evita ataques de *Clickjacking*, impidiendo que la web se cargue dentro de *iframes* externos.
 - **X-Content-Type-Options:** evita ataques relacionados con tipos MIME, haciendo que el navegador respete el tipo de contenido indicado.
 - **X-XSS-Protection:** ayuda a detectar y bloquear ataques XSS en navegadores antiguos.

```
server {  
    listen 80 default_server;  
    listen [::]:80 default_server;  
  
    add_header X-Frame-Options "SAMEORIGIN";  
    add_header X-Content-Type-Options "nosniff";  
    add_header X-XSS-Protection "1; mode=block";
```

Imagen 6: Configuración headers

- **Verificación post-bastionado:** Una nueva iteración de auditoría confirmó la ausencia de metadatos de versión y la presencia efectiva de las nuevas cabeceras de seguridad en la respuesta HTTP.



```

alex@kali: ~
$ curl -I http://192.168.1.34
HTTP/1.1 200 OK
Server: nginx
Date: Thu, 13 Aug 2026 20:53:05 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Wed, 29 Jul 2026 11:15:49 GMT
Connection: keep-alive
ETag: "6a69e0e5-267"
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Accept-Ranges: bytes

```

Imagen 7: Verificación versión oculta

Problemas y Resoluciones durante el Proyecto

- **Error 255/EXCEPTION en Fail2ban:** Durante la creación de la cárcel personalizada (custom-web), el servicio Fail2ban generó repetidos errores de sintaxis y excepciones al intentar arrancar. La resolución requirió el uso del modo de depuración (fail2ban-client -d) y el ajuste milimétrico del archivo jail.local para asegurar que el parámetro filter apuntara correctamente al archivo de configuración del filtro.
- **Fallo de Coincidencia (Match) en Expresiones Regulares de Fail2ban:** Las pruebas iniciales de fuerza bruta con Hydra no eran detectadas por el HIPS debido a que la herramienta inyectaba el nombre de usuario (admin) en los logs de acceso de Nginx, rompiendo el patrón estándar de doble guion esperado por Fail2ban. La solución consistió en utilizar la herramienta fail2ban-regex contra los logs reales para calibrar una expresión regular permisiva (failregex = ^<HOST> - .*) que cazara de forma agnóstica cualquier petición generada por la herramienta de ataque.
- **Error de Mapeo Unicode en ModSecurity:** Tras la activación del motor WAF, Nginx fallaba al reiniciar, indicando un error en la ruta del archivo unicode.mapping. El inconveniente se subsanó localizando el archivo requerido en el repositorio fuente de ModSecurity y copiándolo manualmente al directorio de configuración /etc/nginx/modsec/, ajustando posteriormente la directiva SecUnicodeMapFile en el archivo de configuración.

4. DESPLIEGUE DE LA ESTRATEGIA DE DEFENSA EN PROFUNDIDAD

La estrategia defensiva del laboratorio se estructuró en tres capas complementarias (*Defense in Depth*), garantizando que si un vector de ataque logra eludir el cortafuegos perimetral, existan controles adicionales en las capas de aplicación y host.

Atacante → Suricata (red) → ModSecurity (web) → Fail2ban (sistema) → Servidor

4.1. HIPS a Nivel de Host (Fail2ban)

Para contrarrestar ataques de volumen, escaneo rápido de directorios y fuerza bruta, se desplegó un Sistema de Prevención de Intrusiones en Host (HIPS) mediante **Fail2ban** en el servidor Ubuntu de la DMZ. Fail2ban analiza las bitácoras de acceso en tiempo real y, al detectar anomalías, inyecta reglas de bloqueo directamente en la tabla de filtrado del kernel (iptables).

Configuración e Implementación:

1. **Filtro Personalizado (/etc/fail2ban/filter.d/custom-web.conf):**

Se definió una firma de inspección orientada a capturar cualquier petición entrante procesada por Nginx.



```
GNU nano 7.2 /etc/fail2ban/filter.d/custom-web.conf
[Definition]
failregex = ^<HOST> - .*
ignoreregex =
```

Imágen 8: Filtro web

2. **Cálculo de Cárcel (/etc/fail2ban/jail.local):**

Se creó la cárcel [custom-web] configurada con un umbral estricto: maxretry = 3 intentos en un intervalo findtime = 60 segundos, aplicando un baneo dinámico (bantime = 600 segundos).



```
[custom-web]
enabled = true
filter = custom-web
port = http,https
logpath = /var/log/nginx/access.log
maxretry = 3
findtime = 60
bantime = 600
```

Imágen 9: Configuración cárcel

Inconvenientes Encontrados y Soluciones Aplicadas:

- **Fallo de Coincidencia (0 matched):** Inicialmente, la herramienta fail2ban-regex no reconocía las líneas del archivo /var/log/nginx/access.log generadas durante los ataques con Hydra. Al analizar las trazas reales:

```
192.168.1.33 - admin [11/Aug/2026:22:37:14 +0000] "GET / HTTP/1.1" 200 615 "-"
"Mozilla/4.0 (Hydra)"
```

- Se identificó que el ataque inyectaba el nombre de usuario (- admin) rompiendo la expresión regular estándar que esperaba dos guiones (- -). Además, una sintaxis previa provocó un fallo de parsing (código de error 255).
- **Solución:** Se ajustó el filtro a la expresión canónica agnóstica failregex = ^<HOST> - .*, permitiendo a Fail2ban procesar la IP de origen independientemente de la cabecera o usuario presente en la petición HTTP.

Validación Práctica:

Tras lanzar ráfagas de peticiones desde **Kali Linux** con gobuster e hydra, se ejecutó el control de estado.

El sistema confirmó la captura exitosa con el estado Currently banned: 1 reflejando la IP de Kali. La regla se constató en el kernel mediante `sudo iptables -L f2b-custom-web -n -v` y las pruebas posteriores desde Kali devolvieron un bloqueo total (Connection timed out).

```
(alex@kali)-[~]
└─$ hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.1.34 http-get /
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or
binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-13 16:12:19
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399),
[DATA] attacking http-get://192.168.1.34:80/
[80][http-get] host: 192.168.1.34 login: admin password: password
[80][http-get] host: 192.168.1.34 login: admin password: 123456
[80][http-get] host: 192.168.1.34 login: admin password: 12345
[80][http-get] host: 192.168.1.34 login: admin password: 123456789
[80][http-get] host: 192.168.1.34 login: admin password: iloveyou
[80][http-get] host: 192.168.1.34 login: admin password: princess
[80][http-get] host: 192.168.1.34 login: admin password: 1234567
[80][http-get] host: 192.168.1.34 login: admin password: 12345678
[80][http-get] host: 192.168.1.34 login: admin password: abc123
[80][http-get] host: 192.168.1.34 login: admin password: daniel
[80][http-get] host: 192.168.1.34 login: admin password: babygirl
[80][http-get] host: 192.168.1.34 login: admin password: monkey
[80][http-get] host: 192.168.1.34 login: admin password: lovely
[80][http-get] host: 192.168.1.34 login: admin password: jessica
[80][http-get] host: 192.168.1.34 login: admin password: rockyou
[80][http-get] host: 192.168.1.34 login: admin password: nicole
1 of 1 target successfully completed, 16 valid passwords found
```

Imagen 10: Ataque hydra

```
alex@userver:~$ sudo fail2ban-client status custom-web
Status for the jail: custom-web
|- Filter
|   |- Currently failed: 1
|   |- Total failed:    17
|   \- File list:       /var/log/nginx/access.log
|- Actions
|   |- Currently banned: 1
|   |- Total banned:    1
|   \- Banned IP list:  192.168.1.83
```

Imagen 11: Comprobación de bloqueo

4.2. WAF a Nivel de Aplicación (ModSecurity + OWASP CRS)

Mientras Fail2ban responde al *volumen* de tráfico, se implementó un **Web Application Firewall (WAF)** para evaluar el *contenido* de cada paquete en la Capa 7 del modelo OSI, neutralizando vulnerabilidades del Top 10 de OWASP como Inyecciones SQL (SQLi) y Cross-Site Scripting (XSS).

Configuración e Implementación:

1. **Motor ModSecurity v3:** Se instalaron los módulos libmodsecurity3 y libnginx-mod-http-modsecurity, vinculándolos al bloque server de Nginx.
2. **Motor de Reglas Activo:** Se modificó la directiva inicial en modsecurity.conf pasando de SecRuleEngine DetectionOnly a SecRuleEngine On para habilitar el bloqueo activo.
3. **OWASP Core Rule Set (CRS):** Se clonó e integró el repositorio oficial del conjunto de reglas de OWASP (coreruleset), habilitando la inspección profunda de firmas de exploit.

```
GNU nano 7.2 /etc/nginx/sites-available/default
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    # Habilitar ModSecurity WAF
    modsecurity on;
    modsecurity_rules_file /etc/nginx/modsec/main.conf;

    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";
```

Imagen 12: Configuración WAF en nginx

Inconvenientes Encontrados y Soluciones Aplicadas:

- **Error de Sintaxis en Nginx al Reiniciar:** Al ejecutar `sudo nginx -t`, el servicio falló indicando que no localizaba el archivo de mapeo de caracteres.
- **Solución:** Se descargó y posicionó el archivo oficial `unicode.mapping` dentro del directorio de configuración `/etc/nginx/modsec/`, resolviendo la dependencia requerida por el motor en la línea 211 de `modsecurity.conf`.

Validación Práctica:

Se ejecutaron dos pruebas de concepto (PoC) mediante `curl` desde Kali Linux:

1. Prueba de Inyección SQL:

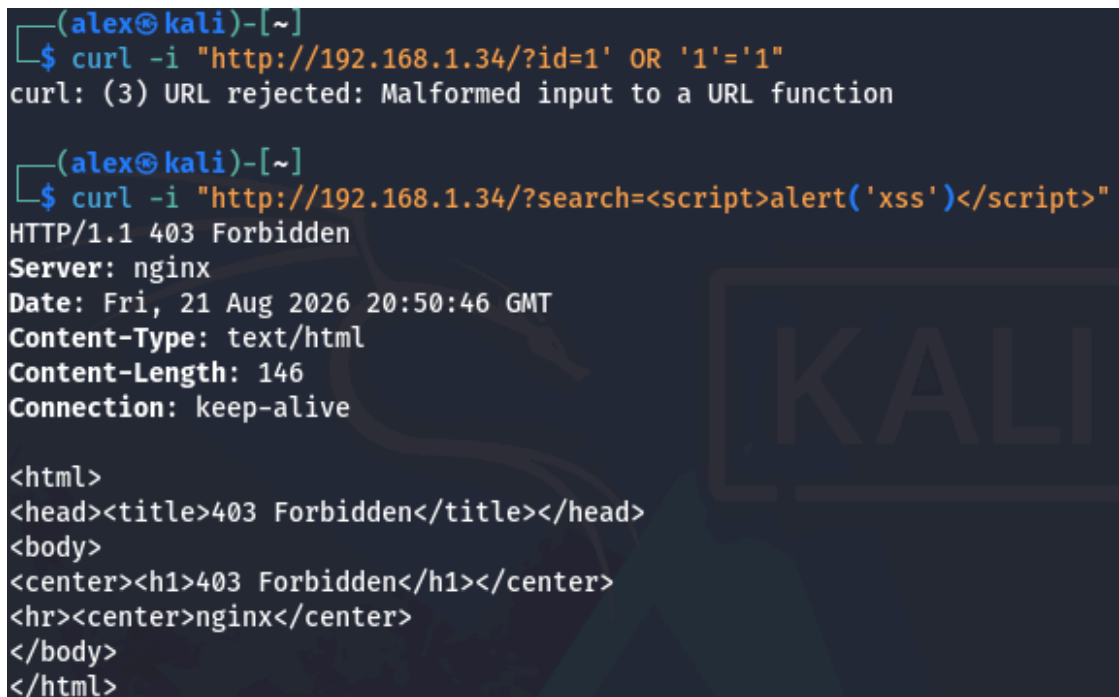
```
curl -i "http://<IP_WAN_PFSense>/?id=1' OR '1'='1"
```

2. *Resultado:* El WAF rechazó la petición en la fase de normalización de cadenas identificando sintaxis maliciosa (*Malformed input*).

3. Prueba de Cross-Site Scripting (XSS):

```
curl -i "http://<IP_WAN_PFSense>/?search=<script>alert('xss')</script>"
```

4. *Resultado:* ModSecurity interceptó el payload devolviendo un código **HTTP/1.1 403 Forbidden**. El registro en `/var/log/nginx/error.log` confirmó la activación de la regla OWASP CRS correspondiente.



```
(alex@kali)-[~]
$ curl -i "http://192.168.1.34/?id=1' OR '1'='1"
curl: (3) URL rejected: Malformed input to a URL function

(alex@kali)-[~]
$ curl -i "http://192.168.1.34/?search=<script>alert('xss')</script>"
HTTP/1.1 403 Forbidden
Server: nginx
Date: Fri, 21 Aug 2026 20:50:46 GMT
Content-Type: text/html
Content-Length: 146
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

Imagen 13: Ataques rechazados

4.3. IPS a Nivel Perimetral (Suricata en pfSense)

Como primera línea de defensa antes de que el tráfico alcance la DMZ, se instaló y configuró el motor **Suricata** actuando como Sistema de Prevención de Intrusiones (IPS) en la interfaz WAN del cortafuegos **pfSense**.

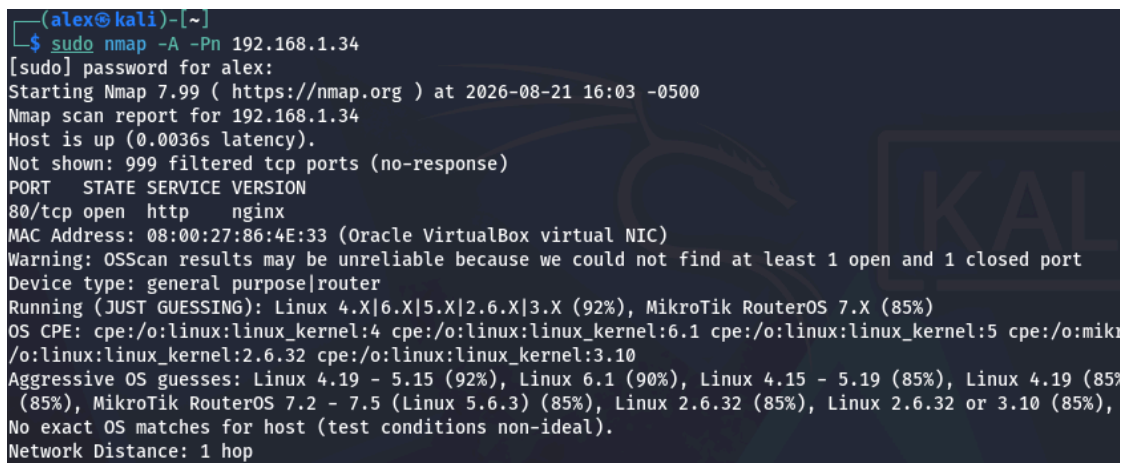
Configuración e Implementación:

1. **Base de Datos de Firmas:** Se desplegó el paquete Suricata desde el gestor de pfSense y se activaron las reglas comunitarias **Emerging Threats Open (ET Open)**.

2. **Modo Preventivo Active:** Se configuró el motor en la interfaz WAN asociando la directiva **Block Offenders**, convirtiendo la herramienta en un IPS capaz de inyectar bloqueos temporales o permanentes en la tabla de filtrado del cortafuegos (snort2c).

Análisis Técnico de Comportamiento y Solución de Dudas:

- **Comportamiento en la Auditoría Inicial:** Durante la primera simulación de reconocimiento lanzada con `sudo nmap -A -Pn <IP_WAN_PFSense>`, Kali detectó el **puerto 80/TCP como abierto**.



```
(alex@kali)-[~]
└─$ sudo nmap -A -Pn 192.168.1.34
[sudo] password for alex:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-21 16:03 -0500
Nmap scan report for 192.168.1.34
Host is up (0.0036s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      nginx
MAC Address: 08:00:27:86:4E:33 (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose|router
Running (JUST GUESSING): Linux 4.X|6.X|5.X|2.6.X|3.X (92%), MikroTik RouterOS 7.X (85%)
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:6.1 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7.1
Aggressive OS guesses: Linux 4.19 - 5.15 (92%), Linux 6.1 (90%), Linux 4.15 - 5.19 (85%), Linux 4.19 (85%),
(85%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (85%), Linux 2.6.32 (85%), Linux 2.6.32 or 3.10 (85%),
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
```

Imagen 14: Puerto 80 abierto

- **Justificación de Diseño:** Este comportamiento es completamente correcto y esperado en un IPS basado en análisis de eventos/firmas. El puerto 80 debe estar visible inicialmente para dar servicio legítimo. Suricata requiere evaluar los primeros paquetes de la ráfaga o los encabezados HTTP para determinar si existe un patrón malicioso. Una vez que la herramienta nmap activó la firma ET SCAN Nmap Scripting Engine, el motor ordenó el bloqueo inmediato.
- **Resultado Definitivo:** En el segundo escaneo consecutivo desde Kali Linux, la totalidad de los puertos aparecieron en estado **ignored / filtered**.

Validación Práctica:

La inspección en la WebGUI de pfSense ratificó el éxito de la prueba:

- **Pestaña *Alerts*:** Muestra de eventos en rojo detallando el intento de escaneo por firmas conocidas desde la IP de Kali.

08/21/2026 21:04:11		3	ICMP	Generic Protocol Command Decode	192.168.1.33 8	192.168.1.34 9	1:2200025  	SURICATA ICMPv4 unknown code
08/21/2026 21:04:10		3	ICMP	Generic Protocol Command Decode	192.168.1.33 8	192.168.1.34 9	1:2200025  	SURICATA ICMPv4 unknown code
08/21/2026 21:04:10		3	ICMP	Generic Protocol Command Decode	192.168.1.33 8	192.168.1.34 9	1:2200025  	SURICATA ICMPv4 unknown code
08/21/2026 21:04:09		3	ICMP	Generic Protocol Command Decode	192.168.1.33 8	192.168.1.34 9	1:2200025  	SURICATA ICMPv4 unknown code
08/21/2026 21:04:07		3	ICMP	Generic Protocol Command Decode	192.168.1.33 8	192.168.1.34 9	1:2200025  	SURICATA ICMPv4 unknown code
08/21/2026 21:04:07		3	ICMP	Generic Protocol Command Decode	192.168.1.33 8	192.168.1.34 9	1:2200025  	SURICATA ICMPv4 unknown code
08/21/2026 21:04:07		3	ICMP	Generic Protocol Command Decode	192.168.1.33 8	192.168.1.34 9	1:2200025  	SURICATA ICMPv4 unknown code
08/21/2026 21:04:06		3	ICMP	Generic Protocol Command Decode	192.168.1.33 8	192.168.1.34 9	1:2200025  	SURICATA ICMPv4 unknown code

Imagen 15: IP bloqueada por Suricata

- **Pestaña *Blocks*:** Inclusión automática de la dirección IP de Kali Linux en la tabla dinámica de bloqueo del perímetro.

5. AUDITORÍA, RESULTADOS Y CONCLUSIONES

5.1. Validación de Pruebas de Concepto (PoC)

Para comprobar la efectividad de la arquitectura de **Defensa en Profundidad**, se ejecutó una batería de auditorías ofensivas desde la estación de pruebas **Kali Linux** (192.168.1.33). Cada ataque fue diseñado para evaluar una capa específica de la infraestructura defensiva.

- **Fingerprinting**: se ocultó la versión de Nginx y se mantuvieron las cabeceras de seguridad.
- **Fuerza bruta**: Fail2ban detectó los intentos y bloqueó la IP.
- **SQL Injection**: ModSecurity rechazó la petición maliciosa.
- **XSS**: ModSecurity bloqueó el ataque con un 403 Forbidden.
- **Escaneo de vulnerabilidades**: Suricata detectó el escaneo y bloqueó las conexiones posteriores.

Síntesis de Evidencias Forenses:

1. **Host-Based Prevention (Fail2ban)**: La ejecución del comando `sudo fail2ban-client status custom-web` confirmó la inclusión de la IP atacante en la lista de baneos, verificándose en la tabla `f2b-custom-web` de iptables.
2. **Application Firewall (ModSecurity)**: La inspección de `/var/log/nginx/error.log` mostró la traza de auditoría donde las reglas 941XXX del OWASP Core Rule Set interceptaron el payload malicioso enviando la denegación 403.
3. **Network IPS (Suricata)**: En la consola WebGUI de pfSense, la pestaña **Alerts** registró la firma ET SCAN Nmap Scripting Engine, forzando la inyección de la IP en la tabla interna `snort2c` de pfSense (**Blocks**).

5.2. Evaluación Global y Líneas Futuras

Evaluación Global

El laboratorio ha demostrado la validez práctica del paradigma **Defensa en Profundidad**. La dependencia exclusiva de un elemento defensivo (por ejemplo, un firewall perimetral tradicional) resulta insuficiente frente a amenazas modernas.

Mediante la integración sinérgica de controles en distintas capas:

- **pfSense + Suricata** aíslan la red frente a reconocimiento agresivo y ataques a nivel de transporte.
- **ModSecurity WAF** filtra el tráfico web legítimo del malicioso en Capa 7 sin afectar la disponibilidad del servicio.
- **Fail2ban** protege los recursos del servidor local contra denegaciones de servicio y saturación de peticiones.
- **Hardening en Nginx** elimina vectores de explotación pasiva asociados a la divulgación de información.

El diseño por zonas (WAN, LAN, DMZ) garantiza que, en el supuesto de que un servicio en la DMZ fuese comprometido, las políticas de aislamiento de pfSense previenen cualquier desplazamiento lateral (*pivot*) hacia la red corporativa interna (LAN).

Líneas Futuras de Trabajo

Para elevar el nivel de madurez de la infraestructura a un entorno de producción corporativo, se proponen las siguientes mejoras técnicas:

1. **Migración a Suricata en Modo Inline (Netmap):**
Sustituir el modo *Legacy* por el modo *Inline* en pfSense. Esto permitirá a Suricata evaluar los paquetes directamente en la cola de la interfaz de red, descartando (*Drop*) el primer paquete malicioso de un escaneo de Nmap sin llegar a responder si el puerto 80 está abierto.
2. **Despliegue de un SIEM Centralizado (Wazuh / Elastic Stack):**
Implementar agentes de monitorización en el servidor Ubuntu y pfSense para reenviar los eventos de Nginx, ModSecurity, Fail2ban y Suricata a un servidor SIEM centralizado. Esto permitirá la correlación de eventos en tiempo real y la creación de un panel de control SOC (*Security Operations Center*).
3. **Implementación de Single Packet Authorization (SPA) / Port Knocking:**
Ocultar por completo los puertos de administración restantes mediante tecnologías de autenticación de paquete único (fwknop), manteniendo los servicios en estado invisible (*DROP*) hasta la validación de una firma criptográfica previa.
4. **Cifrado de Extremo a Extremo (HTTPS / TLS):**
Configurar certificados de clave pública (Let's Encrypt) e implementar terminación SSL/TLS en Nginx/WAF, asegurando la confidencialidad de la información y permitiendo la inspección de tráfico cifrado por parte de ModSecurity.

BIBLIOGRAFÍA

[Defensa en profundidad en 7 minutos | Lo esencial para Security+](#)

[Presentación de la serie sobre la creación de laboratorios virtuales de ciberseguridad: Ep1](#)

[Defense in Depth AI Cybersecurity: Complete Guide 2026](#)

[Defense in Depth \[Beginner's Guide\] | CrowdStrike](#)